

고차 함수 — Java Stream API와 같은 역할

JS 고차 함수는 Java Stream API와 동일한 역할을 한다. 배열을 선언적으로 가공하는 핵심 도구.

위치: `Array.prototype`에 직접 정의 — Java와 달리 별도의 `stream()` 호출 불필요

Java Stream과의 핵심 차이

관점	Java Stream	JS 고차 함수
진입점	<code>collection.stream()</code> 별도 호출	배열 메서드에 직접 존재
결과 수집	<code>.collect(Collectors.toList())</code> 필요	<code>filter/map</code> 결과가 바로 배열
재사용	일회용 (최종 연산 후 재사용 불가)	배열이므로 몇 번이든 재사용 가능
지연 평가	O (최종 연산까지 실행 안 함)	X (즉시 실행)

핵심 메서드

메서드	설명	반환값	시간복잡도
<code>filter(callback)</code>	조건에 맞는 요소만 추출	새 배열	O(n)
<code>map(callback)</code>	각 요소를 변환	새 배열	O(n)
<code>reduce(callback, initial)</code>	모든 요소를 하나로 합침	단일 값	O(n)
<code>find(callback)</code>	조건에 맞는 첫 번째 요소	요소 또는 undefined	O(n)
<code>findIndex(callback)</code>	조건에 맞는 첫 번째 인덱스	인덱스 또는 -1	O(n)
<code>some(callback)</code>	하나라도 조건 만족?	boolean	O(n)
<code>every(callback)</code>	모두 조건 만족?	boolean	O(n)
<code>flat(depth)</code>	중첩 배열 평탄화	새 배열	O(n)
<code>flatMap(callback)</code>	map + flat(1)	새 배열	O(n)
<code>forEach(callback)</code>	각 요소에 대해 실행	undefined	O(n)

패턴

코딩테스트 출제 빈도: 패턴1 중복 제거 + 정렬 > 패턴2 조건 필터링 > 패턴3 변환 > 패턴4 누적 계산 > 패턴5 검색 > 패턴6 조건 확인 > 패턴7 체이닝

패턴 1: 중복 제거 + 정렬 — Set + sort (고차함수 없이 해결)

언제 쓰는가

- 중복을 제거하고 정렬된 결과가 필요할 때
- Java에서는 Stream의 `distinct()` + `sorted()`로 처리하지만, JS에서는 고차함수 없이 Set + sort 조합으로 해결

실무에서는

- 태그 목록 정리, 중복 제거된 검색 추천어

```
const tags = ["java", "python", "java", "c++", "python", "go"];

// Set으로 중복 제거 → 배열로 변환 → 정렬
const uniqueSorted = [...new Set(tags)].sort();
console.log(uniqueSorted); // ["c++", "go", "java", "python"]

// 숫자 배열의 경우 sort 콜백 필수
const nums = [3, 1, 4, 1, 5, 9, 2, 6, 5];
const uniqueNums = [...new Set(nums)].sort((a, b) => a - b);
console.log(uniqueNums); // [1, 2, 3, 4, 5, 6, 9]
```

패턴 2: 조건 필터링 — filter

언제 쓰는가

- 조건에 맞는 요소만 추출할 때

실무에서는

- 검색 필터, 권한별 메뉴 표시

```
const scores = [45, 78, 92, 33, 88, 61, 55];

// 60점 이상만 추출
const passed = scores.filter(score => score >= 60);
console.log(passed); // [78, 92, 88, 61]

// 짝수만 추출
const nums = [1, 2, 3, 4, 5, 6, 7, 8];
const evens = nums.filter(n => n % 2 === 0);
console.log(evens); // [2, 4, 6, 8]

// 특정 문자열 포함하는 것만
const words = ["apple", "application", "banana", "appetite"];
const appWords = words.filter(w => w.startsWith("app"));
console.log(appWords); // ["apple", "application", "appetite"]
```

패턴 3: 변환 — map

언제 쓰는가

- 각 요소를 다른 형태로 변환할 때

실무에서는

- API 응답 → UI 컴포넌트 데이터 변환

```
const names = ["alice", "bob", "charlie"];

// 첫 글자 대문자
const capitalized = names.map(name =>
  name.charAt(0).toUpperCase() + name.slice(1)
);
console.log(capitalized); // ["Alice", "Bob", "Charlie"]

// 객체 배열에서 특정 필드만 추출
const users = [
  { id: 1, name: "Alice", age: 25 },
  { id: 2, name: "Bob", age: 30 },
  { id: 3, name: "Charlie", age: 28 },
];

const userNames = users.map(user => user.name);
console.log(userNames); // ["Alice", "Bob", "Charlie"]

// 숫자 → 문자열 변환
const nums = [1, 2, 3];
const str = nums.map(String);
console.log(str); // ["1", "2", "3"]
```

패턴 4: 누적 계산 — reduce

언제 쓰는가

- 배열 전체를 하나의 값으로 합칠 때

실무에서는

- 장바구니 총합, 통계 집계

```
// 합계
const prices = [1200, 3500, 800, 5000];
const total = prices.reduce((sum, price) => sum + price, 0);
console.log("총합:", total); // 총합: 10500

// 최대값
```

```
const max = prices.reduce((m, price) => Math.max(m, price), -Infinity);
console.log("최대:", max); // 최대: 5000

// 빈도 세기 (reduce로)
const fruits = ["apple", "banana", "apple", "cherry", "banana", "apple"];
const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;
  return acc;
}, {});
console.log(count); // { apple: 3, banana: 2, cherry: 1 }

// 그룹화
const people = [
  { name: "Alice", team: "A" },
  { name: "Bob", team: "B" },
  { name: "Charlie", team: "A" },
  { name: "David", team: "B" },
];

const groups = people.reduce((acc, person) => {
  if (!acc[person.team]) acc[person.team] = [];
  acc[person.team].push(person.name);
  return acc;
}, {});
console.log(groups); // { A: ["Alice", "Charlie"], B: ["Bob", "David"] }
```

패턴 5: 검색 — find / findIndex

언제 쓰는가

- 조건에 맞는 첫 번째 요소가 필요할 때
- Java의 `findFirst()`에 대응하지만, JS에서 더 자주 사용됨

실무에서는

- 특정 ID로 사용자 찾기

```
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" },
  { id: 3, name: "Charlie" },
];

// find - 요소 반환 (없으면 undefined)
const bob = users.find(user => user.id === 2);
console.log(bob); // { id: 2, name: "Bob" }

// findIndex - 인덱스 반환 (없으면 -1)
const idx = users.findIndex(user => user.name === "Charlie");
console.log(idx); // 2
```

```
// 없는 경우
const notFound = users.find(user => user.id === 99);
console.log(notFound); // undefined
```

패턴 6: 조건 확인 — some / every

언제 쓰는가

- "하나라도 만족?" vs "모두 만족?" 확인할 때
- Java의 `anyMatch()` / `allMatch()`에 대응하지만, JS에서 더 자주 사용됨

실무에서는

- 폼 유효성 검증, 전체 선택 체크박스 상태

```
const scores = [85, 92, 78, 95, 88];

// some - 하나라도 100점?
const hasPerfect = scores.some(s => s === 100);
console.log("만점자 있음:", hasPerfect); // 만점자 있음: false

// every - 모두 70점 이상?
const allPassed = scores.every(s => s >= 70);
console.log("전원 합격:", allPassed); // 전원 합격: true

// 폼 유효성 예시
const fields = [
  { name: "email", value: "test@test.com", valid: true },
  { name: "password", value: "1234", valid: false },
];

const isFormValid = fields.every(f => f.valid);
console.log("폼 유효:", isFormValid); // 폼 유효: false
```

패턴 7: 메서드 체이닝 — filter + map + reduce

언제 쓰는가

- 여러 처리를 연속으로 할 때

실무에서는

- 데이터 변환 파이프라인

```
// JS: 60점 이상의 점수를 10점 가산 후 합계
const scores = [45, 78, 92, 33, 88, 61, 55];
```

```
const result = scores
  .filter(s => s >= 60)      // [78, 92, 88, 61]
  .map(s => s + 10)         // [88, 102, 98, 71]
  .reduce((sum, s) => sum + s, 0); // 359

console.log("가산 후 합계:", result); // 가산 후 합계: 359
```

```
// Java: 동일한 로직
// import java.util.*;
// import java.util.stream.*;
//
// List<Integer> scores = Arrays.asList(45, 78, 92, 33, 88, 61, 55);
//
// int result = scores.stream()
//   .filter(s -> s >= 60)      // [78, 92, 88, 61]
//   .map(s -> s + 10)         // [88, 102, 98, 71]
//   .reduce(0, Integer::sum);  // 359
//
// System.out.println("가산 후 합계: " + result); // 가산 후 합계: 359
```

Java는 `stream()` 진입 + `collect()` 또는 `reduce()` 종결이 필요하지만, JS는 배열 메서드를 바로 체이닝하면 된다.

주의사항

`forEach`는 `undefined`를 반환한다 — 체이닝 불가

```
const arr = [1, 2, 3];

// 잘못된 사용: forEach 뒤에 체이닝
// arr.forEach(n => n * 2).filter(n => n > 2); // TypeError!

// forEach는 부수 효과(side effect)용도로만 사용
arr.forEach(n => console.log(n)); // 1, 2, 3
```

`map`에서 `return`을 빼먹으면 `undefined` 배열

```
const nums = [1, 2, 3];

// 중괄호를 쓰면 return 필수
const wrong = nums.map(n => { n * 2 }); // [undefined, undefined, undefined]
const right = nums.map(n => { return n * 2 }); // [2, 4, 6]
```

```
const best = nums.map(n => n * 2); // [2, 4, 6] - 한 줄이면 중괄호 생략
```

reduce 초기값 생략의 위험

```
// 초기값 없으면 첫 번째 요소가 초기값
const nums = [1, 2, 3];
const sum = nums.reduce((acc, n) => acc + n); // 6 - 동작은 하지만...

// 빈 배열에서 초기값 없으면 에러!
// [].reduce((acc, n) => acc + n); // TypeError: Reduce of empty array
// with no initial value

// 항상 초기값을 명시하는 것이 안전
const safeSum = [].reduce((acc, n) => acc + n, 0); // 0
```

cf. 고차함수와 수학의 합성함수

같은 점: 둘 다 함수를 대상으로 한다. 일반 함수가 숫자/문자열을 다루듯, 함수 자체를 입력이나 출력으로 사용한다.

다른 점: 합성함수 $f(g(x))$ 는 함수의 출력을 다른 함수의 입력으로 넣는 한 가지 패턴이다. 고차함수는 그보다 범위가 넓어서, 함수를 인자로 받거나(`map(callback)`), 함수를 반환하는 함수 전체를 포함한다. 즉 합성함수는 고차함수의 부분집합이다.